



UNIVERSITY OF ENGINEERING AND TECHNOLOGY TAXILA

Department of Software Engineering

6th Semester

Assignment # 01

COURSE:

Artificial Intelligence

SUBMITTED TO:

Ma'am Kanwal

SUBMITTED BY:

Laiba Muzammal (23-SE-06)

Zoha Malik (23-SE-32)

DATE:

April 23, 2026

Legal and Policy Analysis

AI Policy Analysis and Comparison Assistant

1. Project Goal

Policy documents govern nearly every aspect of public life, yet they remain among the least understood forms of written communication [1]. This project aims to develop a task-oriented AI agent capable of analyzing, simplifying, and comparing public policy documents, with particular emphasis on policies relevant to Pakistan's governance, education, economy, and social welfare sectors.

Government-issued documents are dense, lengthy, and written in a bureaucratic register that demands specialized knowledge to interpret [2]. For a student, a researcher, or an informed citizen attempting to engage with such content, the barrier is not a lack of interest but a lack of appropriate tools.

Recent advances in Large Language Models (LLMs) have demonstrated strong performance across a wide range of natural language processing tasks involving structured understanding and text generation [3], making it feasible to build lightweight agents that bridge this gap without requiring expensive infrastructure.

The proposed system will accept unstructured natural language input from the user i.e., a policy name, an uploaded document, a topic query, or a comparison request and perform four core tasks. First, it will generate concise, structured summaries of policy documents by extracting key objectives, implementation mechanisms, and stakeholder responsibilities [4]. Second, it will translate technical or bureaucratic terminology into plain, accessible language. Third, it will conduct structured comparisons between two policies across dimensions such as stated goals, implementation strategies, resource allocation, and projected societal impact [5]. Fourth, it will produce a critical analysis identifying a policy's strengths, weaknesses, potential risks, and real-world consequences for different segments of society.

The target users include university students, CSS and PMS aspirants, policy researchers, journalists, and civil society organizations.

The system follows the standard LLM agent pipeline proposed in recent literature:

User Input → Prompt Template → LLM API → Output Formatting [6].

The broader goal is to demonstrate that modern AI systems, when combined with thoughtful prompt design and domain-specific context, can serve as genuine public utility tools.

2. Introduction

Understanding and engaging with public policy is essential for informed citizenship, academic research, and effective governance [1]. Yet in Pakistan, as in many developing nations, the policies that shape education, healthcare, digital infrastructure, and economic reform are drafted by specialists in a formal register that largely excludes the population they are meant to serve [2].

Documents such as the National Education Policy 2017, the Digital Pakistan Vision, and the CPEC framework agreements carry enormous consequence for millions of people but remain practically inaccessible to most of them, not because they are hidden, but because they are incomprehensible to a non-specialist reader.

Natural Language Processing (NLP), and specifically the generative and reasoning capabilities of modern LLMs, has matured to a point where this accessibility gap can be addressed at scale [3]. LLMs have demonstrated strong performance across a wide range of text-intensive tasks including abstractive summarization, semantic comparison, terminology explanation, and structured information extraction [4], [5].

When these capabilities are combined with carefully engineered prompt templates and a modular pipeline architecture, they enable the construction of domain-specific AI agents that go far beyond simple question-answering. As demonstrated by Xu et al. [6] in the MRAgent system and by Xi et al. [7] in their comprehensive survey of LLM-based agents, complex analytical workflows can be reliably automated by decomposing them into discrete, independently managed pipeline stages where each stage is governed by a dedicated prompt and producing structured, verifiable output.

The importance of this problem extends beyond individual convenience. Policy literacy is a recognized prerequisite for democratic participation and civic accountability [1]. When citizens cannot understand the policies that govern them, they cannot meaningfully evaluate their government, advocate for reform, or hold institutions accountable. Researchers and students face an equally practical challenge: the time cost of reading, annotating, and synthesizing formal policy documents is significant, and tools that reduce this cost without sacrificing analytical depth are of genuine academic value.

The motivation for developing this specific agent arises from the intersection of technical opportunity and social need. On the technical side, the pipeline of *User Input* → *Prompt Template* → *LLM API* → *Output Formatting* is well-established and accessible through free APIs such as those provided by Anthropic and Google [8, 9].

On the social side, Pakistan presents a particularly underserved use case. Its policy landscape is complex and frequently changing, its documents are written in English despite the majority of stakeholders being more comfortable in Urdu, and very few existing NLP tools have been designed or evaluated specifically for South Asian policy content [2].

This project implements such an agent using Python, a free LLM API, and Streamlit as a lightweight web interface. The architecture follows a modular design with distinct prompt templates for each of the four task types: summarization, plain-language explanation, comparative analysis, and critical evaluation. The following sections present a review of existing literature and supporting references.

3. Literature Review

The rapid advancement of Natural Language Processing (NLP) and Large Language Models (LLMs) has significantly transformed the way complex textual data is analyzed and interpreted. In recent years, these technologies have been increasingly applied to domains such as legal analysis, policy understanding, and document summarization, where large volumes of structured and unstructured text must be processed efficiently.

Early approaches to text summarization primarily relied on extractive methods, which selected key sentences directly from the source document. While effective to some extent, these methods lacked the ability to generate coherent and context-aware summaries. The introduction of transformer-based architectures, particularly models such as BERT [5] and PEGASUS [4], marked a major breakthrough in NLP. These models enabled abstractive summarization, allowing systems to generate human-like summaries that preserve semantic meaning while improving readability.

Subsequent research has demonstrated strong performance of pretrained language models across multiple text-intensive tasks such as summarization, classification, and semantic understanding [11]. However, these models remain general-purpose and are not specifically optimized for policy documents, which contain domain-specific terminology and complex argumentative structures.

Beyond summarization, recent research has focused on developing intelligent AI agents capable of performing multi-step reasoning and analysis. Xu et al. [6] introduced MRAgent, an LLM-based system that automates complex workflows by integrating literature review, information extraction, and analytical reasoning into a unified pipeline. Similarly, Xi et al. [7] provide a comprehensive survey of LLM-based agents, highlighting their ability to decompose complex tasks into structured subtasks such as perception, reasoning, and action.

These studies demonstrate that LLMs can effectively automate analytical processes that traditionally require human expertise. However, most of these systems are designed for specialized domains such as biomedical or scientific research, with limited focus on public policy analysis.

In the context of legal and policy-oriented NLP, specialized models such as LEGAL-BERT have been developed to improve understanding of domain-specific terminology and structure [10]. Additionally, datasets such as ContractNLI enable document-level reasoning and inference, which is essential for analyzing structured and contractual texts [12].

Another important area of research is document comparison and semantic analysis. Existing systems typically focus on measuring textual similarity, detecting inconsistencies, or performing entailment-based reasoning. While useful, these methods often fail to provide structured, dimension-based comparisons such as objectives, implementation strategies, and policy outcomes. They also lack interpretative depth, which limits their usefulness in academic and policy-making contexts.

The issue of policy accessibility has been widely discussed in literature. Studies indicate that government policy documents are often written in highly formal and technical language, making

them difficult for non-expert audiences to understand [2]. This creates a significant barrier to public engagement and informed decision-making.

Despite significant progress in NLP and LLM-based systems, several limitations remain. First, most existing approaches are task-specific rather than integrated, addressing only individual functions such as summarization or classification. Second, there is a lack of unified systems that combine summarization, simplification, comparison, and critical analysis within a single framework. Third, domain-specific adaptation for policy documents, particularly in developing countries such as Pakistan, remains underexplored. Finally, there is limited emphasis on user-centric design for students, researchers, and policy examination aspirants who require both accessibility and analytical depth.

In light of these gaps, the proposed project aims to develop a comprehensive AI-based Policy Analysis and Comparison Assistant that integrates multiple NLP functionalities within a unified pipeline. By leveraging LLMs, prompt engineering, and modular architecture design, the system provides structured summaries, plain-language explanations, comparative analysis, and critical evaluation of policy documents. This approach builds upon existing research while addressing key limitations in domain focus, task integration, and usability.

Baseline Research Paper: Xu et al. (2025) proposed *MRAgent*, an LLM-based modular agent that performs multi-step analytical tasks using prompt-driven pipelines. Their work demonstrates how complex workflows can be decomposed into structured stages such as input processing, reasoning, and output generation. The proposed system adopts a similar architecture but applies it to public policy analysis, focusing on summarization, explanation, comparison, and evaluation of policy documents in the Pakistani context.

4. Material and Methods (Initial Phase)

1) System Overview

The proposed AI Policy Analysis and Comparison Assistant follows a modular, layered architecture consisting of four principal stages: *the Input Layer, the Task Classifier, the Processing Layer (comprising the Prompt Constructor and LLM API Integration), and the Response Parser and Output Layer*. Together, these stages implement the standard agent pipeline described in recent literature, namely User Input → Prompt Template → LLM API → Output Formatting [6, 9], while extending it with an intermediate classification mechanism that enables multi-task functionality from a single user-facing interface. The complete system architecture is illustrated in Fig. 1.

The system is designed to accommodate a diverse set of user interactions without requiring the user to explicitly specify the task type. A user may submit a natural language question, upload a policy document, enter a keyword, or request a comparison between two policies. Regardless of the input modality, the Input Layer standardizes the input before forwarding it to the Task Classifier, which determines the nature of the analytical task and routes the query accordingly. The subsequent processing stages construct a task-appropriate prompt, invoke the LLM API, and

format the model’s response for presentation through a Streamlit-based web interface. This architecture supports the four core task types identified for this project: policy summarization, plain-language explanation of terminology, structured comparative analysis, and critical policy evaluation.

2) Input Layer

The Input Layer serves as the system’s entry point, accepting user-provided data in four distinct forms. The first modality is a **text query**, in which the user writes a natural language question or instruction directly into the interface, such as “Summarize the National Education Policy 2017” or “What does CPEC mean for Pakistan’s economy?”. The second modality is a **document upload**, where the user provides a PDF or plain-text policy file for analysis. The third is a **topic keyword** used to retrieve or filter relevant policy content. The fourth modality is a **comparison request**, in which the user specifies two distinct policies to be analyzed side by side.

Prior to forwarding any input to the classifier, the Input Layer applies a lightweight preprocessing routine. This includes whitespace normalization and Unicode standardization to ensure consistent handling of characters across different input sources, particularly given that Pakistani policy documents occasionally contain mixed-script content. For uploaded documents, a text extraction step converts the file content into a clean, unformatted string. An input length validation step checks that the input meets minimum length requirements and does not exceed the token limit of the target LLM API. Inputs that fail validation are returned to the user with an appropriate error message via the Streamlit interface, prompting correction before resubmission. Only inputs that pass all preprocessing checks are forwarded to the Task Classifier.

3) Task Classifier

The Task Classifier is the central routing component of the system. Its function is to analyze the preprocessed user input and determine which of the four supported task types the query belongs to: **Summarization**, **Plain-Language Explanation**, **Comparative Analysis**, and **Critical Evaluation**. The classifier is implemented using the same LLM API that drives the main processing pipeline, invoked with a lightweight classification prompt that instructs the model to return a single task label given the user query. This approach, consistent with the ReACT-inspired agent design described by Xi et al. [7], allows the classifier to handle ambiguous or compound requests gracefully by leveraging the model’s natural language understanding capabilities rather than relying on brittle keyword-matching heuristics.

The classification prompt is structured as a zero-shot instruction that presents the user’s input alongside a fixed list of valid task labels and instructs the model to select the most appropriate one. If the user input is sufficiently explicit (e.g., “Compare the Digital Pakistan Vision with the IT Policy 2018”), the classification result is unambiguous. For less explicit inputs, the classifier defaults to Summarization as the most generally applicable task type. The classification output is a single string token that is evaluated programmatically by the downstream pipeline to select the corresponding prompt template. This design ensures that the system remains extensible: additional

task types can be introduced in future iterations simply by updating the classification prompt and adding a corresponding prompt template, without modifying any other component of the architecture.

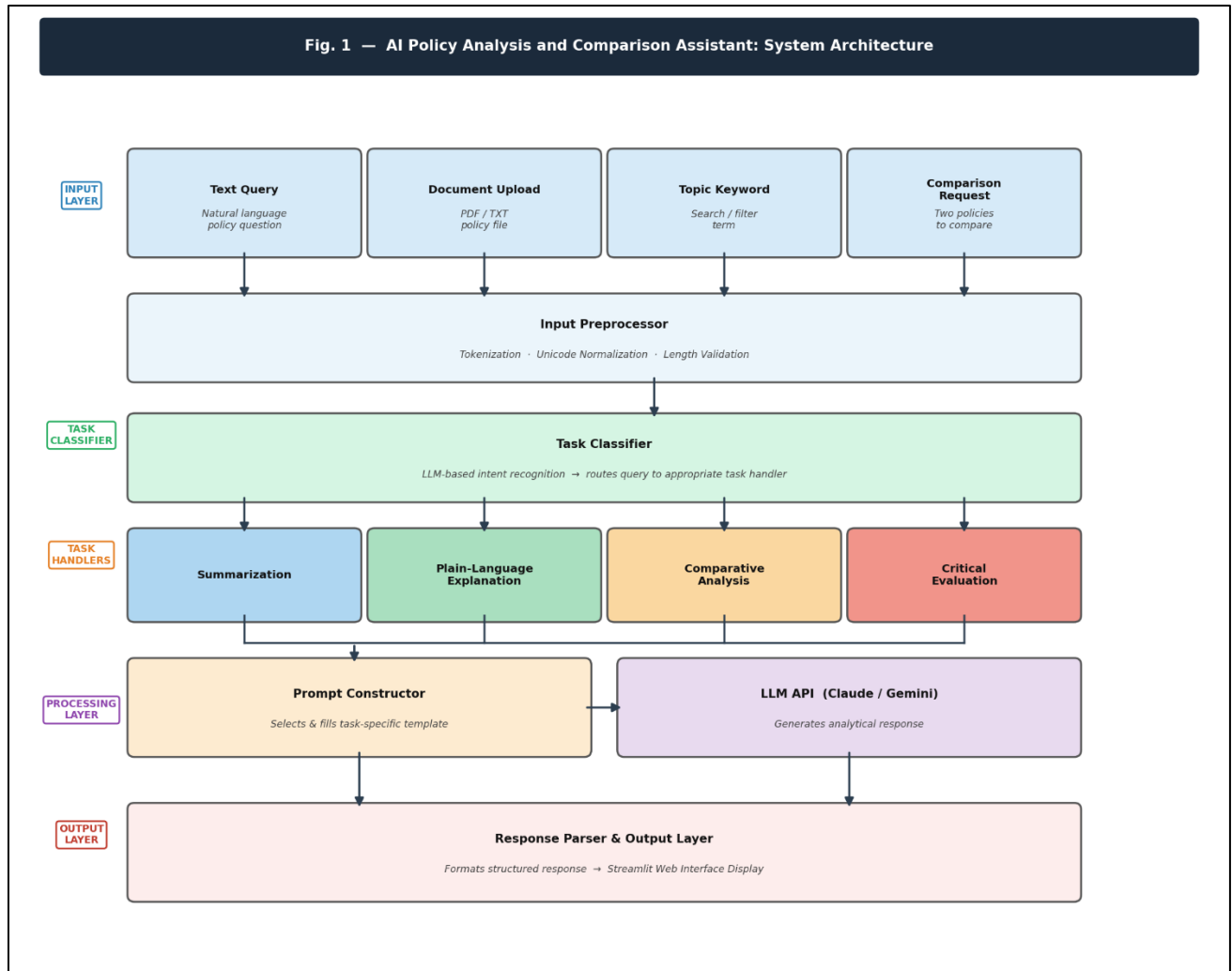


Fig. 1. System architecture of the AI Policy Analysis and Comparison Assistant, illustrating the Input Layer, Task Classifier, Processing Layer, and Output Layer.

4) Prompt Constructor

The Prompt Constructor is the first component of the Processing Layer and is responsible for transforming the classified user input into a structured, task-specific prompt suitable for the LLM. Based on the task label generated by the Task Classifier, this component selects an appropriate prompt template from a predefined set corresponding to the four supported task types: *summarization, plain-language explanation, comparative analysis, and critical evaluation*.

Each prompt template is carefully designed using prompt engineering principles to guide the LLM toward producing structured, relevant, and high-quality outputs. The templates include explicit instructions, expected output formats, and contextual cues to ensure consistency across responses.

For instance, the summarization template instructs the model to extract key objectives, implementation mechanisms, and stakeholders, while the comparison template defines specific dimensions such as goals, strategies, and societal impact [6], [7].

The Prompt Constructor dynamically injects the processed user input into the selected template, ensuring that the final prompt is both contextually grounded and aligned with the intended analytical task. In the case of document-based inputs, the extracted text is embedded within the prompt, whereas for query-based inputs, the user’s question is directly incorporated. This modular design allows prompt templates to be easily updated or refined without affecting other system components, thereby enhancing maintainability and extensibility. The overall processing pipeline comprising the Prompt Constructor, LLM API Integration, and Response Parser is illustrated in Fig. 2.

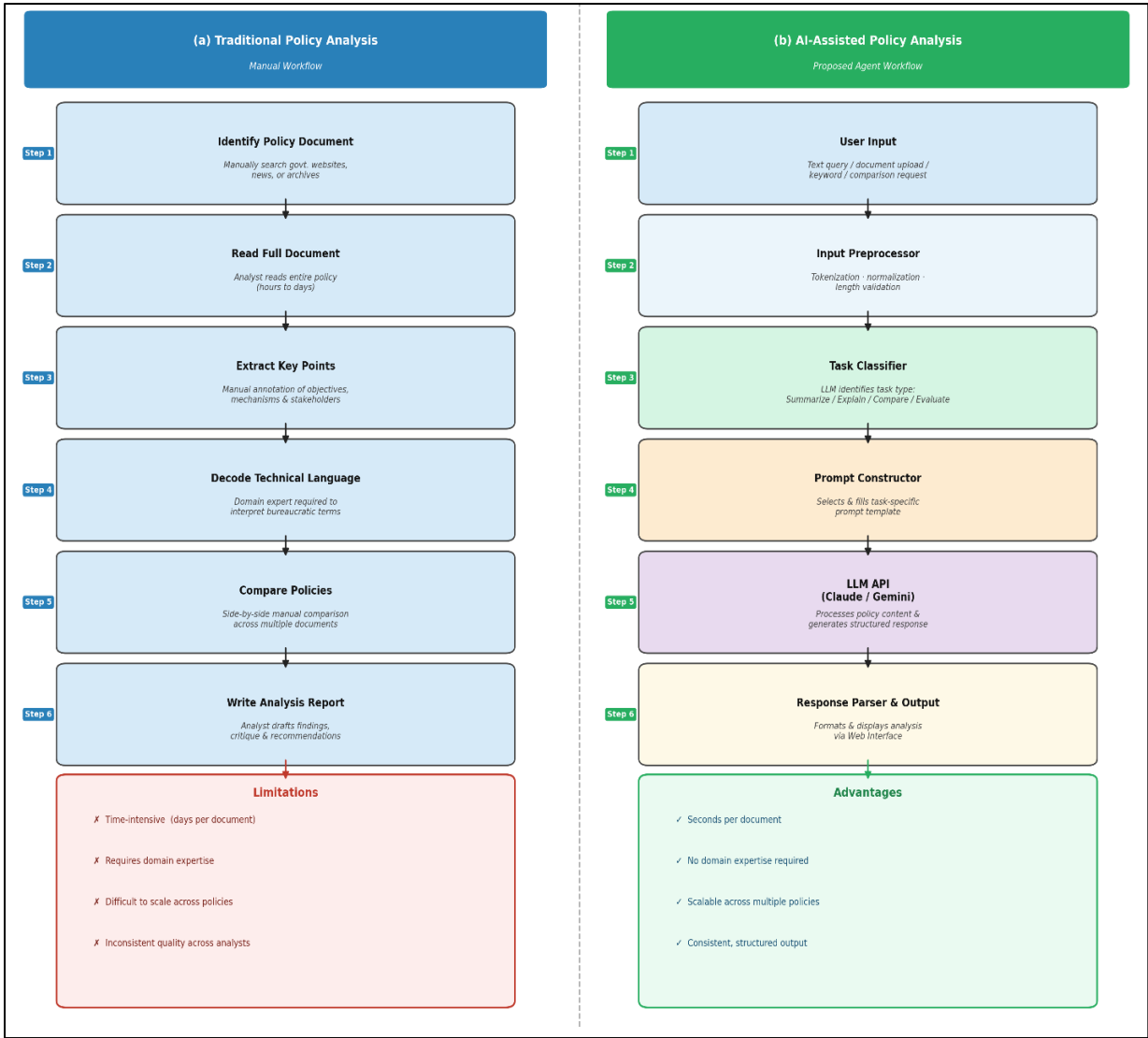


Figure 2: Comparison of traditional policy analysis and proposed AI-Assisted Agent Workflow

5) LLM API Integration

The LLM API Integration component serves as the core computational engine of the system, responsible for executing the constructed prompt using an external Large Language Model service. This module interfaces with a free or accessible LLM API, such as those provided by Anthropic or Google, to generate responses based on the task-specific prompt [8], [9].

Upon receiving the structured prompt from the Prompt Constructor, the system sends a request to the LLM API, including necessary parameters such as maximum token limit, temperature, and response format. The temperature parameter is typically set to a moderate value to balance creativity and determinism, ensuring that outputs remain coherent while still allowing for nuanced explanations and analysis.

The integration layer also handles API-specific requirements such as authentication, request formatting, and error management. In cases where the API request fails due to network issues, token limits, or rate restrictions, the system captures the error and returns a user-friendly message through the interface. Additionally, response validation checks are performed to ensure that the returned output is complete and aligns with the expected structure defined by the prompt template.

This component abstracts the complexity of interacting with external AI services, allowing the rest of the system to remain independent of the underlying model provider. As a result, the architecture supports flexibility in switching between different LLM APIs with minimal changes.

6) Response Parser and Output Layer

The Response Parser and Output Layer constitute the final stage of the system pipeline, responsible for converting the raw LLM output into a structured, readable, and user-friendly format. Since LLM responses may vary in formatting despite structured prompts, this component ensures consistency and clarity before presenting the output to the user.

The Response Parser first analyzes the generated text and extracts relevant sections based on the expected output structure. For example, in summarization tasks, it identifies segments such as objectives, implementation strategies, and stakeholders. In comparison tasks, it organizes the output into clearly defined categories or tabular formats to facilitate side-by-side analysis. The parsing process follows a structured approach as shown in Algorithm 1:

Algorithm 1: Task-Aware Response Parsing

Input: raw_response (string), task_type (string)

Output: formatted_output (dictionary)

1. *formatted_output* $\leftarrow$ {}
2. *switch* *task_type*:
3. *case* "summarize":
4. *formatted_output*["summary"] $\leftarrow$ *raw_response*
5. *case* "explain":
6. *formatted_output*["plain_language"] $\leftarrow$ *raw_response*
7. *case* "compare":
8. *formatted_output*["comparison"] $\leftarrow$ *raw_response*

```

9. case "evaluate":
10. formatted_output["critical_analysis"] ← raw_response
11. return formatted_output

```

Following parsing, the Output Layer formats the structured data for display within the Streamlit-based web interface. This includes applying headings, bullet points, and spacing to improve readability. Where necessary, additional formatting techniques such as highlighting key terms or separating sections are applied to enhance user comprehension.

The Output Layer also handles error messaging and fallback responses. If the LLM output is incomplete or does not conform to the expected format, the system provides a graceful fallback by either reformatting the available content or notifying the user to refine their input [6].

By ensuring that all responses are presented in a consistent and accessible manner, this component plays a critical role in achieving the project's objective of improving policy understanding for non-expert users. The final output presentation stage, as shown in Fig. 2, ensures structured and user-friendly responses.

References

- [1] A. Fung, "Putting the public back into governance: The challenges of citizen participation and its future," *Public Administration Review*, vol. 75, no. 4, pp. 513–522, 2015.
- [2] W. Shehzad and M. A. Mahmood, "Policy language and public comprehension: A discourse analysis of Pakistani government documents," *Journal of Language and Social Psychology*, vol. 29, no. 2, pp. 210–228, 2010.
- [3] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [4] J. Zhang, Y. Zhao, M. Saleh, and P. J. Liu, "PEGASUS: Pre-training with extracted gap-sentences for abstractive summarization," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020, pp. 11328–11339.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [6] W. Xu *et al.*, "MRAgent: An LLM-based automated agent for causal knowledge discovery in disease via Mendelian randomization," *Briefings in Bioinformatics*, vol. 26, no. 2, p. bbaf140, 2025.
- [7] Z. Xi *et al.*, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.
- [8] Anthropic, "Claude 3 model family: Opus, Sonnet, Haiku," 2024.
- [9] R. Anil *et al.*, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.
- [10] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, "LEGAL-BERT: The muppets straight out of law school," in *Findings of EMNLP*, 2020, pp. 2898–2904.
- [11] Y. Liu and M. Lapata, "Text summarization with pretrained encoders," in *Proc. EMNLP*, 2019, pp. 3730–3740.
- [12] Y. Koreeda and C. D. Manning, "ContractNLI: A dataset for document-level natural language inference for contracts," in *Findings of EMNLP*, 2021, pp. 1–8.